

Chapter 17. Python from Zero

Install it, understand it, and write your first script that does a real job.

Approved chapter v1

Approved 18 September 2026

Every code block run and its output checked

Contents

Chapter 17. Python from Zero

Why this matters

In plain English

17.1 What a program is, and when to use one

17.2 Installing Python, VS Code, and Jupyter

17.3 Virtual environments and packages

17.4 Your first Python

17.5 Variables and types

17.6 Lists and tuples

17.7 Dictionaries and sets

17.8 Conditions: making decisions

17.9 Loops

17.10 Functions

17.11 Files and folders

17.12 Errors, tracebacks, and debugging

17.13 The standard library and packages

17.14 From notebook to script

17.15 Getting unstuck

Common mistakes and how to spot them

In the real world: the fifteen-minute rescue

Tools

The project: summarize a folder of exports

Timed challenge: the folder in forty minutes

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 17. Python from Zero

Part II — The Analyst

Chapter at a glance

You will learn to: say what a program is and when Python beats a spreadsheet or SQL · install Python, VS Code, and Jupyter on Windows, macOS, or Linux · create a virtual environment for every project and install packages into it · work with variables, strings, numbers, and booleans · use lists, dictionaries, tuples, and sets, and know which to reach for · write conditions and loops, and read a list comprehension · write functions with arguments, defaults, and return values · read and write CSV, JSON, and text files with `pathlib` and the standard library · read a traceback, handle errors on purpose, and debug a script · use the standard library and `pip` · turn a notebook experiment into a script someone else can run · get unstuck without waiting for anyone.

Before you start: Chapters 1–9 for the ideas, Chapter 6 for installing tools, and Chapter 14 for what cleaning involves. No programming experience is assumed. If you've written formulas in Excel (Chapters 10–11) or SQL (Chapters 12–13), you already think the way this chapter needs.

Time needed: 25–30 hours, spread over three to four weeks. Programming is learned by typing, not by reading.

Tools: Python 3.13 or 3.14 (any 3.10+ works for everything here), VS Code, and Jupyter. A laptop: phones and tablets aren't enough from this chapter on.

Practice data: `companion/ch17/` , built by `build_ch17_files.py` from the full Riverstone dataset: `sales_exports/` (twelve monthly CSV files of the 24 key accounts' 2025 order lines, plus a README so your code has to ignore non-CSV files), `products.json` , `targets_2025.csv` , and `broken_export.csv` for the error-handling section.

Why this matters

You can already do a great deal without programming. Chapters 10 and 11 made spreadsheets do real work; Chapters 12 and 13 made a database answer questions no spreadsheet could; Chapter 16 turned the answers into a report that refreshes itself.

Python is for the things those tools can't do, and for the things they can do only once.

- **Anything the tool can't do at all:** call an API, read a folder of 200 files, rename 5,000 images, scrape a page, fit a model, run a simulation.
- **Anything you'd otherwise repeat:** the monthly cleaning in Chapter 14 was already a recorded pipeline; a Python script does the same job, runs on a schedule, logs what it did, and emails the result (Chapters 18 and 20).

- **Anything that needs to be exact and reviewable:** code in a file can be read, versioned (Chapter 26), and tested. A sequence of clicks can't.

It also changes what you're allowed to say yes to. "Can you check every branch export for the last three years?" is a week of clicking or twenty minutes of Python. That difference is most of the distance between an analyst and a senior analyst, and all of the distance toward the data-engineering and data-science work in Parts III and IV.

The chapter starts from zero: what a variable is, what a loop is, why your program says `IndentationError`. If you've programmed before, skim to section 17.11 and do the exercises.

In plain English

A recipe is a program. It has **ingredients** (data), **steps in order** (statements), and **instructions that depend on something** ("if the dough is sticky, add flour" is a condition; "knead for ten minutes" is a loop). It has **names** for things ("the sauce" is a variable), and **sub-recipes** you can reuse ("make the dough" is a function).

The difference is that a cook fills in the gaps and a computer doesn't. Tell a cook "add the milk" and they'll use the milk on the counter. Tell a computer to add the milk and it will ask which milk, in what amount, and stop if it can't find it. That's not the computer being stupid; it's the computer being exact, which is the whole reason it can do the same job on 200 files without getting bored or making a typo in row 4,000.

So programming is mostly two skills:

1. **Saying exactly what you mean**, in an order that works.
2. **Reading the complaint** when you didn't. Error messages are not failure; they're the computer telling you which step confused it.

Everything else in this chapter is vocabulary.

17.1 What a program is, and when to use one

A **program** is a set of instructions a computer follows. **Python** is a language for writing them: it reads almost like English, it's the most common language in data work, and it comes with a large collection of ready-made tools.

Where Python fits next to what you know

Job	Best tool	Why
Look at a few hundred rows, try a formula	Spreadsheet	Immediate, visual, no setup
Aggregate millions of rows that live in a database	SQL	The database is built for it

Job	Best tool	Why
A report several people read every month	Power BI	Model once, refresh forever
Clean and combine 40 files with different formats	Python	Repeatable, and it scales past one machine's patience
Anything on a schedule with logging and alerts	Python	Scripts can run unattended (Chapter 20)
Statistics, forecasting, machine learning	Python	The libraries are there (Parts IV–V)
Call an API, scrape a page, talk to another system	Python	Nothing else in the analyst's kit can

Python doesn't replace the others. Most real work is a chain: SQL pulls, Python cleans and combines, Power BI or Excel presents.

Three ways to run Python, and what each is for

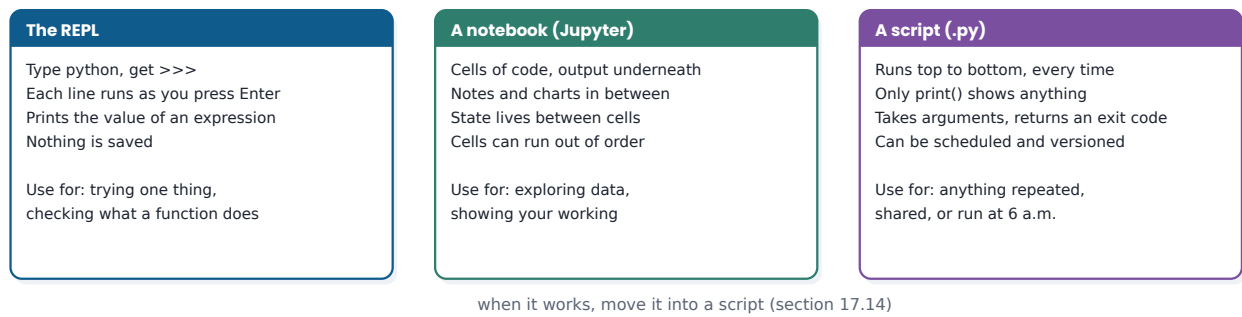


Figure 17.1 – Explore in a notebook, deliver a script.

The three ways you'll run Python

1. **The REPL** (type `python` in a terminal): a prompt where each line runs as you press Enter. Good for trying one thing.
2. **A notebook** (Jupyter, or a `.ipynb` file in VS Code): cells of code with their output underneath, mixed with notes. Good for exploring data and for showing your working.
3. **A script** (a `.py` file you run with `python script.py`): the whole thing, top to bottom, every time. Good for anything repeated, scheduled, or shared.

Analysts explore in a notebook and deliver a script. Section 17.14 shows the move from one to the other.

17.2 Installing Python, VS Code, and Jupyter

Chapter 6 set up the toolkit; here's the detail, and what to do when it goes wrong.

Python

- **Windows:** download the installer from python.org and tick "Add python.exe to PATH" on the first screen before clicking Install. That one checkbox prevents the most common Windows problem. (The Microsoft Store version also works; avoid having both.)
- **macOS:** the python.org installer, or `brew install python@3.13` if you use Homebrew. The `python3` that ships with macOS is old and used by the system: don't build on it.
- **Linux:** usually installed. `sudo apt install python3 python3-venv python3-pip` on Ubuntu or Debian adds the parts you need.

Check it:

```
python --version      # Windows
python3 --version     # macOS and Linux
```

You want **3.13 or 3.14** (the latest stable release when this was written). Anything from 3.10 runs every example in this chapter.

Watch out: `python` versus `python3`. On macOS and Linux, `python` may not exist or may point at an old version; use `python3` and `pip3`. On Windows, `python` is right, and the `py` launcher (`py -3.13`) picks a version when several are installed. This chapter writes `python`; translate as needed.

VS Code

Install VS Code, then its **Python** extension (Microsoft) and **Jupyter** extension. Useful settings on day one: **View** → **Terminal** for the built-in terminal, **Ctrl+`** to toggle it, and **Ctrl+Shift+P** for the command palette, where **Python: Select Interpreter** chooses which Python (and which virtual environment) a folder uses.

Jupyter

`pip install jupyterlab` inside your project's virtual environment (next section), then `jupyter lab` opens it in a browser. Or open a `.ipynb` file in VS Code and run cells there, which is what most analysts do now.

When installation fights back

Symptom	Cause	Fix
<code>python: command not found</code> or <code>'python' is not recognized</code>	Not on PATH	Reinstall with "Add to PATH" ticked, or use the full path; on macOS/Linux try <code>python3</code>
<code>pip: command not found</code>	Same, or pip missing	<code>python -m pip</code> always works; <code>python -m ensurepip</code> installs it
The wrong version runs	Several Pythons installed	<code>python -m</code> with an explicit version, or <code>py -3.13</code> on Windows; check Select Interpreter in VS Code
Permission denied when installing	Installing into the system Python	Use a virtual environment (never <code>sudo pip install</code>)

Symptom	Cause	Fix
Company laptop blocks the installer	IT policy	Ask IT, or use a cloud notebook (Google Colab) until you have a machine you control

17.3 Virtual environments and packages

A **package** is code someone else wrote that you can use: `pandas` for tables, `requests` for web calls, `openpyxl` for Excel files. `pip` installs them. The trap is installing everything into one shared Python: two projects that need different versions of the same package will fight, and one day something that worked stops working.

A **virtual environment** is a private copy of Python for one project, with its own installed packages.

```
cd path/to/my-project
python -m venv .venv           # create it, once per project
.venv\Scripts\activate        # activate: Windows
source .venv/bin/activate      # activate: macOS and Linux
python -m pip install pandas   # installs into this project only
python -m pip freeze > requirements.txt
deactivate                    # when you're done
```

The prompt shows `(.venv)` when it's active. In VS Code, **Python: Select Interpreter** → pick the one inside `.venv`, and new terminals activate it for you.

`requirements.txt` records exactly what's installed, so a colleague (or you, next year, on another machine) can recreate it with `python -m pip install -r requirements.txt`. Commit it to Git (Chapter 26); never commit the `.venv` folder itself.

Tool note: other environment managers. `conda` (Anaconda or Miniconda) manages environments and non-Python dependencies, and is common in science teams and on locked-down corporate machines. `uv` and `poetry` are newer and faster. They all solve the same problem; learn `venv` first, because it's built in and every explanation you'll read online assumes it.

17.4 Your first Python

Open a terminal, activate an environment, and type `python`. You get a `>>>` prompt. Try:

```
print("Hello, Riverstone")
2 + 2 * 10
```

```
Hello, Riverstone
```

The REPL prints the value of an expression, but a script doesn't: in a `.py` file, only `print()` shows anything. That surprises everyone once.

A first script. Create `hello.py` in VS Code:

```
customer = "Metro Mart"
orders = 14
print("Customer:", customer)
print("Orders this year:", orders)
print(f"{customer} placed {orders} orders, about {orders / 12:.1f} a month.")
```

```
Customer: Metro Mart
Orders this year: 14
Metro Mart placed 14 orders, about 1.2 a month.
```

Run it with `python hello.py`, or press the ► button in VS Code.

Three things to notice:

- **`print()` writes to the screen.** Separate several values with commas, and Python puts spaces between them.
- **An f-string** (`f"...{expression}..."`) inserts values into text. `:.1f` formats a number with one decimal place. F-strings are the clearest way to build text in modern Python.
- **# starts a comment.** Everything after it on that line is for humans.

17.5 Variables and types

A **variable** is a name for a value: `orders = 14` means "from now on, `orders` refers to 14". The `=` is assignment, not equality (that's `==`).

Every value has a **type**, and the type decides what you can do with it.

```
name = "Metro Mart"           # str: text
orders = 14                    # int: whole number
revenue = 284530.50            # float: number with decimals
is_key_account = True          # bool: True or False
rep = None                     # NoneType: "no value"

print(type(name), type(orders), type(revenue), type(is_key_account), type(rep))
print(orders * 2, revenue / orders, name.upper(), not is_key_account)
```

```
<class 'str'> <class 'int'> <class 'float'> <class 'bool'> <class 'NoneType'>
28 20323.60714285714 METRO MART False
```

Naming

Names may use letters, digits, and underscores, and can't start with a digit. The convention is `lower_case_with_underscores`. Good names save more time than any other habit: `net_revenue` beats `nr`, and `customers_without_city` beats `list2`.

Strings

```
customer = " metro mart "  
clean = customer.strip().title()  
print(clean, len(clean))  
print(clean.replace("Mart", "Market"), clean.startswith("Metro"), "Mart" in clean)  
print("2025-01-15".split("-"), "-".join(["2025", "01", "15"]))
```

```
Metro Mart 10  
Metro Market True True  
['2025', '01', '15'] 2025-01-15
```

These are the same operations as Chapter 14's cleaning, one value at a time: `strip()` is TRIM, `title()` and `upper()` are case functions, `replace()` is SUBSTITUTE, `split()` is Text to Columns.

Numbers

```
quantity, unit_price, discount_pct = 45, 430, 5  
net = quantity * unit_price * (1 - discount_pct / 100)  
print(net, round(net, 2), quantity // 10, quantity % 10, 2 ** 10)  
print(int("45") + 5, float("430.50"), str(net) + " rupees")
```

```
18382.5 18382.5 4 5 1024  
50 430.5 18382.5 rupees
```

`//` is integer division (how many whole cartons of 10), `%` is the remainder, `**` is power. `int()`, `float()`, and `str()` convert between types, and they raise an error rather than guess: `int("twenty")` fails, which is exactly what you want in a cleaning script.

Watch out: floats are approximate. `0.1 + 0.2` is `0.30000000000000004` in Python, as in every language that uses binary floating point, and as in Excel, which hides it by rounding what it displays. For money, round when you present, compare with a tolerance, and use the `decimal` module when exactness matters (accounting systems, tax).

```
print(0.1 + 0.2, 0.1 + 0.2 == 0.3, round(0.1 + 0.2, 2) == 0.3)
```

```
0.30000000000000004 False True
```

17.6 Lists and tuples

A **list** holds several values in order. It's the workhorse of Python.

```
cities = ["Mumbai", "Pune", "Bengaluru", "Delhi"]
print(len(cities), cities[0], cities[-1], cities[1:3])

cities.append("Kolkata")
cities.insert(0, "Ahmedabad")
cities.remove("Delhi")
print(cities)
print(sorted(cities), "Pune" in cities, cities.index("Pune"))
```

```
4 Mumbai Delhi ['Pune', 'Bengaluru']
['Ahmedabad', 'Mumbai', 'Pune', 'Bengaluru', 'Kolkata']
['Ahmedabad', 'Bengaluru', 'Kolkata', 'Mumbai', 'Pune'] True 2
```

- **Indexing starts at 0.** `cities[0]` is the first item; `cities[-1]` is the last.
- **Slicing** `cities[1:3]` takes items 1 and 2: the start is included, the end isn't. It reads oddly for a day and then becomes natural.
- **`sorted()` returns a new list;** `cities.sort()` changes the list in place and returns `None`. Assigning the result of `.sort()` to a variable is a classic beginner bug.

Numbers in lists:

```
line_values = [2900.0, 20900.0, 12470.0, 6450.0, 38700.0]
print(sum(line_values), max(line_values), min(line_values), round(sum(line_values) /
len(line_values), 2))
print(sorted(line_values, reverse=True)[:3])
```

```
81420.0 38700.0 2900.0 16284.0
[38700.0, 20900.0, 12470.0]
```

A **tuple** is a list that can't be changed: `("Mumbai", "West")`. Use one for a fixed pair or record, and for a function returning several values. Tuples can be unpacked:

```
city, region = ("Mumbai", "West")
print(city, region)
```

```
Mumbai West
```

17.7 Dictionaries and sets

A **dictionary** maps keys to values: a lookup table, the same idea as XLOOKUP's two columns (Chapter 11) or a mapping table in SQL (Chapter 14, section 14.5).

```
city_region = {"Mumbai": "West", "Pune": "West", "Bengaluru": "South", "Kolkata": "East"}
print(city_region["Mumbai"], len(city_region))
print(city_region.get("Jaipur"), city_region.get("Jaipur", "Unknown"))

city_region["Delhi"] = "North"          # add or update
for city, region in city_region.items():
    print(f"{city:<10} {region}")
```

```
West 4
None Unknown
Mumbai    West
Pune      West
Bengaluru South
Kolkata    East
Delhi      North
```

- **dict[key]** raises **KeyError** if the key is missing; **.get(key, default)** returns a default instead. In cleaning code, **.get()** with an explicit default is usually what you want.
- **.keys()**, **.values()**, **.items()** give you the parts; **for k, v in d.items()** is the standard way to loop.
- **f"{city:<10}"** pads text to ten characters, which is how you line up a printed table.

Counting with a dictionary, the pattern behind every "group by" you'll ever write by hand:

```
statuses = ["Delivered", "Delivered", "Cancelled", "Shipped", "Delivered", "Cancelled"]
counts = {}
for status in statuses:
    counts[status] = counts.get(status, 0) + 1
print(counts)
```

```
{'Delivered': 3, 'Cancelled': 2, 'Shipped': 1}
```

A **set** holds unique values, unordered, and answers "is it in there?" quickly:

```
segments = {"Retail", "Wholesale", "Retail", "Hospitality"}
print(sorted(segments), len(segments))
a, b = {"Mumbai", "Pune", "Delhi"}, {"Delhi", "Kolkata"}
print(sorted(a & b), sorted(a | b), sorted(a - b))
```

```
['Hospitality', 'Retail', 'Wholesale'] 3
['Delhi'] ['Delhi', 'Kolkata', 'Mumbai', 'Pune'] ['Mumbai', 'Pune']
```

& is intersection (in both), **|** is union (in either), **-** is difference (in the first only). Sets have no order, so printing one directly gives an order that can change between runs; **sorted()** makes the output predictable. Set difference is the fastest way to answer "which customer codes are in the export but not in the master list?", the Python version of Chapter 12's anti-join.

Which collection should I use?

list	Ordered, changeable order lines in a file; the months of the year	<code>values[0] · append() · sorted()</code>
tuple	Fixed, unchangeable (city, region); what a function returns	<code>city, region = pair</code>
dict	Lookup: key → value city → region; product → price; counts	<code>d[k] · d.get(k, default) · d.items()</code>
set	Unique, fast membership distinct cities; codes in A but not B	<code>a & b · a b · a - b</code>
DataFrame	Rows and columns (Chapter 18) a whole CSV, a query result	<code>df.groupby() · df.merge()</code>

Figure 17.2 — The four built-in collections, and where a DataFrame takes over.

Which one do I use?

You need	Use
An ordered collection you'll add to and loop over	list
A fixed record of a few fields	tuple
A lookup from one value to another	dict
Unique values, or fast membership tests	set
Rows and columns of data	a DataFrame (Chapter 18)

17.8 Conditions: making decisions

```

net_revenue = 284530.50
target = 250000

if net_revenue >= target:
    status = "above target"
elif net_revenue >= target * 0.9:
    status = "close"
else:
    status = "below target"

print(f"{net_revenue:,.0f} is {status}")

```

```
284,530 is above target
```

The rules:

- **The colon and the indentation are the syntax.** Python has no `END IF`; the indented block is the body. Four spaces is the convention, and VS Code does it for you. Mixing tabs and spaces causes `IndentationError`, which is why editors are configured to insert spaces.
- **`elif`** is "else if", and you can have as many as you like. **`else`** is optional.

- **Comparison operators:** `==` equal, `!=` not equal, `<`, `<=`, `>`, `>=`. Combine with `and`, `or`, `not`, and use brackets when it isn't obvious.

```
segment, orders, city = "Wholesale", 14, None

if segment == "Wholesale" and orders > 10:
    print("bulk buyer")
if city is None:
    print("no city on record")
if not city:
    print("no city: empty string, None, and 0 are all falsy")
print("large" if orders > 20 else "small")
```

```
bulk buyer
no city on record
no city: empty string, None, and 0 are all falsy
small
```

- **`is None`**, not `== None`: `is` tests identity, and `None` is a single object.
- **Truthiness:** empty things (`"`, `[]`, `{}`, `0`, `None`) are false in a condition; everything else is true. Handy, and a trap when `0` is a real value (Chapter 14, section 14.3).
- **The conditional expression** `A if condition else B` fits a small choice on one line.

17.9 Loops

A **loop** repeats work. Almost every Python loop is a `for` loop over a collection.

```
months = ["Jan", "Feb", "Mar"]
for month in months:
    print("processing", month)

for i, month in enumerate(months, start=1):
    print(i, month)

targets = [77350000, 70000000, 108050000]
for month, target in zip(months, targets):
    print(f"{month}: {target/10000000:.2f} crore")
```

```
processing Jan
processing Feb
processing Mar
1 Jan
2 Feb
3 Mar
Jan: 7.74 crore
Feb: 7.00 crore
Mar: 10.80 crore
```

- **`for x in collection:`** takes each item in turn. Nothing to count, nothing to increment.
- **`enumerate()`** gives you the position as well as the item; `start=1` counts like a human.
- **`zip()`** walks two collections together, stopping at the shorter one.

- `range(5)` is 0, 1, 2, 3, 4; `range(1, 13)` is the months. Use it when you need numbers rather than items.

Accumulating a result is the pattern behind most analysis code:

```
line_values = [2900.0, 20900.0, 12470.0, 6450.0, 38700.0]
total = 0
big_lines = []
for value in line_values:
    total += value
    if value > 10000:
        big_lines.append(value)
print(round(total, 2), big_lines, len(big_lines))
```

```
81420.0 [20900.0, 12470.0, 38700.0] 3
```

`while` repeats until a condition stops being true. Analysts need it rarely (retrying a web call, reading until a file ends); a `for` loop over a collection is safer, because it always finishes.

```
attempt = 0
while attempt < 3:
    attempt += 1
    print("attempt", attempt)
    if attempt == 2:
        print("worked")
        break
else:
    print("never ran, because we broke out")
```

```
attempt 1
attempt 2
worked
```

`break` leaves the loop, `continue` skips to the next item, and a loop's `else` runs only if it finished without `break` (rare, but it explains that output).

List comprehensions

A **comprehension** builds a list from another collection in one line. You'll read them constantly, so learn to recognize them even before you write them.

```
values = [2900.0, 20900.0, 12470.0, 6450.0, 38700.0]

doubled = [v * 2 for v in values]
big = [v for v in values if v > 10000]
labels = [f"₹{v:,.0f}" for v in sorted(values, reverse=True)]

print(doubled[:3])
print(big)
print(labels)
```

```
[5800.0, 41800.0, 24940.0]
[20900.0, 12470.0, 38700.0]
['₹38,700', '₹20,900', '₹12,470', '₹6,450', '₹2,900']
```

Read it right to left: for each `v` in `values`, keep it if it's over 10,000, and put `v` in the new list. Dictionaries and sets have the same form: `{k: v for ...}` and `{v for ...}`.

Keep comprehensions to one line and one condition. A comprehension with three nested loops is a puzzle; write the loop out instead.

Watch out: looping when the tool can do it for you. Chapter 18's pandas does whole-column operations without a loop, and a database does them faster still. In plain Python, loops are correct; in pandas, a loop over rows is usually the slow, hard-to-read way. Learn loops first, then learn when not to write them.

17.10 Functions

A **function** is a named piece of code you can run again with different inputs.

```
def net_revenue(quantity, unit_price, discount_pct=0):
    """Return the net value of one order line, after discount."""
    return quantity * unit_price * (1 - discount_pct / 100)

print(net_revenue(45, 430, 5))
print(net_revenue(10, 290))
print(net_revenue(quantity=20, unit_price=1400, discount_pct=12))
```

```
18382.5
2900.0
24640.0
```

- **def name(parameters):** defines it; the indented block is the body; **return** sends a value back. Without `return`, a function returns `None`.
- **Default values** (`discount_pct=0`) make arguments optional.
- **Keyword arguments** (`quantity=20`) make a call self-explaining, and let you skip the order.
- **The docstring** (the string on the first line) is the function's documentation. `help(net_revenue)` prints it.

Why bother, when you could paste the formula three times? Because the formula changes. When Riverstone adds a rebate band (Chapter 11), you change one function, not eleven places, and you can test it:

```
def rebate_pct(annual_value):
    """Riverstone's loyalty rebate: 0% standard, 1% from ₹250,000, 2% from ₹400,000."""
    if annual_value >= 400000:
        return 2
    if annual_value >= 250000:
        return 1
    return 0

for value in [180000, 250000, 399999, 400000]:
    print(f"{value:>7,} → {rebate_pct(value)}%")
```

```
180,000 → 0%
250,000 → 1%
399,999 → 1%
400,000 → 2%
```

Functions have their own **scope**: names created inside a function disappear when it returns, and a function can read names from outside but shouldn't rely on it. Pass what you need in, return what you produce.

```
def summarize(values):
    """Return (count, total, average) for a list of numbers."""
    if not values:
        return 0, 0.0, 0.0
    return len(values), sum(values), sum(values) / len(values)

count, total, average = summarize([2900.0, 20900.0, 12470.0])
print(count, total, round(average, 2))
print(summarize([]))
```

```
3 36270.0 12090.0
(0, 0.0, 0.0)
```

A function returning several values is really returning one tuple, which you can unpack. Handling the empty case explicitly, as here, prevents a `ZeroDivisionError` when a month has no orders.

17.11 Files and folders

Analyst code spends most of its life reading files. Python's `pathlib` handles paths, and the standard library reads the formats.

Paths

```
from pathlib import Path

folder = Path("sales_exports")
print(folder.exists(), folder.is_dir())

files = sorted(folder.glob("*.csv"))
print(len(files), files[0].name, files[0].suffix, files[0].stat().st_size)
print([f.name for f in files[:3]])
```

```
True True
12 riverstone_2025_01.csv .csv 1439
['riverstone_2025_01.csv', 'riverstone_2025_02.csv', 'riverstone_2025_03.csv']
```

- `Path("sales_exports")` is a path object, and `/` joins paths: `folder / "riverstone_2025_01.csv"` works on Windows, macOS, and Linux alike. Never build paths by gluing strings with `\` or `/`.
- `glob("*.csv")` lists matching files; `rglob` searches subfolders too. That's how the README.txt in the folder stays out of your way.

- `.name`, `.stem`, `.suffix`, `.parent`, `.stat().st_size` answer the usual questions.

Text files

```
readme = Path("sales_exports/README.txt")
text = readme.read_text(encoding="utf-8")
print(text.splitlines()[0])

with open("scratch.txt", "w", encoding="utf-8") as f:
    f.write("first line\n")
    f.write("second line\n")

with open("scratch.txt", encoding="utf-8") as f:
    for line_no, line in enumerate(f, start=1):
        print(line_no, line.strip())
```

```
Monthly order-line exports for Riverstone's 24 key accounts, 2025.
1 first line
2 second line
```

- **with open(...) as f:** opens the file and closes it automatically, even if something fails. Always use `with`.
- **Always pass `encoding="utf-8"`**. Without it, Python uses the machine's default, which differs between Windows and everything else, and Indian names with accents or a ₹ sign then read as gibberish on someone else's laptop.
- **Modes:** `"r"` read (the default), `"w"` write (replaces the file), `"a"` append, `"newline=""` when writing CSV.

CSV files

```
import csv

with open("sales_exports/riverstone_2025_01.csv", encoding="utf-8", newline="") as f:
    reader = csv.DictReader(f)
    rows = list(reader)

print(len(rows))
print(rows[0]["customer_name"], rows[0]["net_revenue"], type(rows[0]["net_revenue"]))
print(sorted(rows[0].keys())[4:])
```

```
15
Patel Kitchenware 2900.0 <class 'str'>
['city', 'customer_name', 'discount_pct', 'net_revenue']
```

`csv.DictReader` gives each row as a dictionary keyed by the header, which is much easier to read than numeric positions. **Every value arrives as text**, exactly as in Chapter 14's staging tables: converting is your job, and that's the point.

```
total = 0.0
for row in rows:
    if row["status"] != "Cancelled":
        total += float(row["net_revenue"])
print(f"January net revenue: ₹{total:,.2f} from {len(rows)} lines")
```

```
January net revenue: ₹202,640.00 from 15 lines
```

Writing a CSV:

```
summary = [{"month": "2025-01", "lines": len(rows), "net_revenue": round(total, 2)}]
with open("month_summary.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["month", "lines", "net_revenue"])
    writer.writeheader()
    writer.writerows(summary)

print(Path("month_summary.csv").read_text(encoding="utf-8").strip())
```

```
month,lines,net_revenue
2025-01,15,202640.0
```

`newline=""` prevents the blank line between rows that Windows otherwise adds.

JSON

JSON is the format APIs speak (Chapter 2). It maps onto Python's dictionaries and lists directly.

```
import json

products = json.loads(Path("products.json").read_text(encoding="utf-8"))
print(len(products), products[0]["name"], products[0]["unit_price"])

by_name = {p["name"]: p["unit_price"] for p in products}
print(by_name["Industrial Crate"])

Path("price_list.json").write_text(json.dumps(by_name, indent=2), encoding="utf-8")
print(Path("price_list.json").read_text(encoding="utf-8").splitlines()[:3])
```

```
8 Storage Box 10L 430.0
1400.0
['{' , ' "Storage Box 10L": 430.0, ' , ' "Storage Box 25L": 750.0, ']
```

`json.loads` reads text into Python objects, `json.dumps` turns Python objects into text, and `json.load` / `json.dump` do the same with an open file.

17.12 Errors, tracebacks, and debugging

Your program will fail, often, and that's normal. The skill is reading the complaint.

Reading a traceback

```
values = [1, 2, 3]
print(values[5])
```

```
IndexError: list index out of range
```

A **traceback** prints the chain of calls that led to the error, then the error type and message on the last line. Read it **bottom up**: the last line says *what* went wrong (`IndexError: list index out of range`), and the lines above say *where* (file, line number, and the code). In a long traceback, the last line of your own code, not the library's, is usually where the problem is.

How to read a traceback

```
Traceback (most recent call last):
  File "summarize.py", line 41, in <module>
    raise SystemExit(main(folder))
  File "summarize.py", line 28, in main
    s = summarize(path)
  File "summarize.py", line 19, in summarize
    values.append(float(row["net_revenue"]))
ValueError: could not convert string to float: ''
```

- ← 4. The chain that led there (top = oldest)
- ← 3. Your file and line number
- ← 2. Then this: the line that failed
- ← 1. Read this first: what went wrong

What it tells you here:

a blank `net_revenue` in the CSV reached `float()`. Fix by converting defensively and counting what failed (section 17.12).

Figure 17.3 — Read a traceback from the bottom up: what went wrong, then where.

The errors you'll meet in your first month

Error	Means	Usual cause
<code>SyntaxError</code>	Python can't parse it	Missing <code>:</code> , unbalanced bracket or quote, <code>=</code> instead of <code>==</code>
<code>IndentationError</code>	The blocks don't line up	Mixed tabs and spaces, or a missing indent after <code>:</code>
<code>NameError</code>	No such name	Typo, or using a variable before creating it
<code>TypeError</code>	Wrong type for the operation	<code>"5" + 5</code> , calling something that isn't a function
<code>ValueError</code>	Right type, impossible value	<code>int("twenty")</code> , <code>float("")</code>
<code>KeyError</code>	No such key in a dictionary	A column name that isn't in the CSV header
<code>IndexError</code>	Position doesn't exist	Off-by-one, or an empty list
<code>AttributeError</code>	The object has no such method	<code>list.push()</code> , or a <code>None</code> where you expected an object
<code>FileNotFoundError</code>	The path is wrong	Relative path from the wrong folder, a typo, or the file isn't there yet
<code>ZeroDivisionError</code>	Division by zero	An empty group, a month with no orders
<code>ModuleNotFoundError</code>	Package not installed here	Wrong virtual environment, or you haven't installed it

Two examples worth meeting on purpose:

```
row = {"customer_name": "Metro Mart", "net_revenue": ""}
print(float(row["net_revenue"]))
```

```
ValueError: could not convert string to float: ''
```

```
print(row["city"])
```

```
KeyError: 'city'
```

Both are Chapter 14 problems in Python form: a blank where a number should be, and a column that isn't there.

Handling errors on purpose

```
def to_float(text, default=None):
    """Convert text to a number, or return default when it can't be converted."""
    try:
        return float(text)
    except (ValueError, TypeError):
        return default

print(to_float("430.5"), to_float(""), to_float("twenty", 0.0), to_float(None))
```

```
430.5 None 0.0 None
```

- **try: ... except SomeError:** runs the risky code and catches the failure.
- **Catch what you expect**, not everything. A bare `except:` hides typos, interrupts, and real bugs.
- **else** runs when nothing failed; **finally** runs either way (closing things, logging).
- **Sometimes the right answer is to stop.** Quietly turning bad data into zero is how wrong reports get published; count the failures and report them, exactly as Chapter 14 quarantined rows.

```
from pathlib import Path
import csv

bad_rows = []
clean_total = 0.0
with open("broken_export.csv", encoding="utf-8", newline="") as f:
    for line_no, row in enumerate(csv.DictReader(f), start=2):
        value = to_float(row["net_revenue"])
        quantity = to_float(row["quantity"])
        if value is None or quantity is None:
            bad_rows.append((line_no, row["order_id"], row["quantity"], row["net_revenue"]))
            continue
        clean_total += value

print(f"clean total ₹{clean_total:,.2f}")
print(f"{len(bad_rows)} row(s) could not be read:")
for r in bad_rows:
    print("  line", r[0], "order", r[1], "quantity", repr(r[2]), "value", repr(r[3]))
```

```
clean total ₹278,007.50
1 row(s) could not be read:
  line 7 order 10099 quantity 'twenty' value ''
```

That's the whole shape of a cleaning script: read, convert defensively, keep a list of what failed, report it.

Debugging

When code runs but gives the wrong answer:

1. **Print the thing you assume.** `print(type(x), x)` at the point of doubt answers most questions in seconds. (`repr(x)` shows quotes and escapes, so you can see the trailing space.)
2. **Shrink the problem.** Run it on one file, one row, one function. If a function is wrong, call it with a value whose answer you know.
3. **Check your assumptions about the data**, not only the code: is the column named what you think? Is it text? Does it have blanks?
4. **Use the debugger.** In VS Code, click in the margin to set a breakpoint (a red dot), press **F5**, and step through with **F10** while watching variables. Ten minutes learning it saves hours of print statements.
5. **Rubber-duck it.** Explain the code aloud, line by line, to a colleague or an inanimate object. You'll usually find it yourself in the explaining.
6. **Change one thing at a time**, and keep a working version (Chapter 26's version control is the grown-up form of this).

17.13 The standard library and packages

Python ships with a large **standard library**: no installation needed, only `import`. The modules an analyst uses first:

Module	For
<code>pathlib</code>	Paths and folders
<code>csv</code> , <code>json</code>	Reading and writing those formats
<code>datetime</code>	Dates, times, and differences
<code>statistics</code>	Mean, median, standard deviation
<code>collections</code>	<code>Counter</code> , <code>defaultdict</code> for counting and grouping
<code>re</code>	Regular expressions (Chapter 14's patterns)
<code>math</code>	Rounding, logs, floors
<code>os</code> , <code>sys</code>	The environment, arguments, exit codes
<code>logging</code>	Messages from a script that runs unattended
<code>zipfile</code> , <code>shutil</code>	Archives and file copying

```
from datetime import date, datetime, timedelta

order_date = datetime.strptime("15-01-2025", "%d-%m-%Y").date()
print(order_date, order_date.year, order_date.strftime("%d %b %Y"))
print(order_date + timedelta(days=45), (date(2025, 12, 31) - order_date).days)
```

```
2025-01-15 2025 15 Jan 2025
2025-03-01 350
```

`strptime` parses text into a date using the format codes (`%d` day, `%m` month, `%Y` four-digit year, `%b` short month name); `strftime` formats a date back into text. Those are the same day-first and year-first formats Chapter 14 had to untangle, and the same rule applies: state the format, never let the computer guess.

```
from collections import Counter, defaultdict

statuses = ["Delivered", "Delivered", "Cancelled", "Shipped", "Delivered", "Cancelled"]
print(Counter(statuses))
print(Counter(statuses).most_common(2))

by_city = defaultdict(list)
for city, value in [("Mumbai", 2900), ("Pune", 20900), ("Mumbai", 12470)]:
    by_city[city].append(value)
print(dict(by_city), {c: sum(v) for c, v in by_city.items()})
```

```
Counter({'Delivered': 3, 'Cancelled': 2, 'Shipped': 1})
[('Delivered', 3), ('Cancelled', 2)]
{'Mumbai': [2900, 12470], 'Pune': [20900]} {'Mumbai': 15370, 'Pune': 20900}
```

`Counter` is the counting dictionary from section 17.7, written for you. `defaultdict(list)` creates an empty list the first time you touch a key, which is the shortest correct way to group values by something.

```
import statistics

values = [2900.0, 20900.0, 12470.0, 6450.0, 38700.0]
print(round(statistics.mean(values), 2), statistics.median(values), round(statistics.stdev(values), 2))
```

```
16284.0 12470.0 14266.83
```

Packages beyond the standard library

`pip install` fetches from **PyPI**, the Python Package Index. The ones this book uses:

Package	For	Chapter
<code>pandas</code>	Tables, the analyst's core tool	18
<code>matplotlib</code> , <code>seaborn</code>	Charts	18
<code>openpyxl</code> , <code>xlsxwriter</code>	Reading and writing Excel files	18
<code>requests</code>	Calling APIs	18

Package	For	Chapter
<code>SQLAlchemy</code> , <code>psycopg2</code> , <code>mysql-connector-python</code>	Talking to databases	18
<code>python-dotenv</code>	Keeping credentials out of code	20
<code>jupyterlab</code>	Notebooks	this chapter

Before installing something you found online, check that it's maintained (recent releases, open issues answered), that the name is spelled exactly right (typo-squatting is a real attack), and that it's in your project's virtual environment rather than the system Python.

Watch out: don't paste credentials into code. Database passwords and API keys belong in environment variables or a `.env` file that is never committed (Chapter 20 and Chapter 26). A password in a script is a password in your Git history forever.

17.14 From notebook to script

Exploration belongs in a notebook: run a cell, look, adjust. But a notebook is a poor delivery format, because cells can be run out of order, hidden state builds up, and nobody can schedule it. When something works, move it into a script.

A useful script has five properties: it says what it does, takes its inputs as arguments, does the work in functions, tells you what happened, and can be run again with the same result.

```

summary_script = '''"""Summarize every CSV export in a folder.

Usage:  python summarize_exports.py sales_exports
"""
import csv
import sys
from pathlib import Path

def read_rows(path):
    """Return the rows of a CSV file as a list of dictionaries."""
    with open(path, encoding="utf-8", newline="") as f:
        return list(csv.DictReader(f))

def summarize(path):
    """Return one summary dictionary for a CSV export."""
    rows = read_rows(path)
    values, bad = [], 0
    for row in rows:
        try:
            if row["status"] != "Cancelled":
                values.append(float(row["net_revenue"]))
        except (ValueError, TypeError, KeyError):
            bad += 1
    return {
        "file": path.name,
        "rows": len(rows),
        "lines_counted": len(values),
        "unreadable": bad,
        "net_revenue": round(sum(values), 2),
    }

def main(folder):
    paths = sorted(Path(folder).glob("*.csv"))
    if not paths:
        print(f"No CSV files found in {folder}")
        return 1
    print(f"{'file':<28}{'rows':>6}{'net revenue':>16}")
    total = 0.0
    for path in paths:
        s = summarize(path)
        total += s["net_revenue"]
        print(f"{s['file']:<28}{s['rows']:>6}{s['net_revenue']:>16,.2f}")
    print(f"{'TOTAL':<28}{'':>6}{total:>16,.2f}")
    return 0

if __name__ == "__main__":
    folder = sys.argv[1] if len(sys.argv) > 1 else "sales_exports"
    raise SystemExit(main(folder))
'''

from pathlib import Path
Path("summarize_exports.py").write_text(summary_script, encoding="utf-8")
print("written", len(summary_script.splitlines()), "lines")

```

written 52 lines

Run it from the folder that holds `sales_exports`:

```
import subprocess, sys
out = subprocess.run([sys.executable, "summarize_exports.py", "sales_exports"],
capture_output=True, text=True)
print(out.stdout)
```

file	rows	net revenue
riverstone_2025_01.csv	15	202,640.00
riverstone_2025_02.csv	20	253,664.00
riverstone_2025_03.csv	18	278,007.50
riverstone_2025_04.csv	21	210,281.50
riverstone_2025_05.csv	29	329,358.75
riverstone_2025_06.csv	22	186,928.00
riverstone_2025_07.csv	29	232,692.25
riverstone_2025_08.csv	28	329,282.00
riverstone_2025_09.csv	40	558,314.75
riverstone_2025_10.csv	38	681,070.75
riverstone_2025_11.csv	35	633,408.00
riverstone_2025_12.csv	35	439,823.50
TOTAL		4,335,471.00

What makes it a script rather than a pile of code:

- **A docstring at the top** says what it does and how to run it.
- **Functions with one job each**, each with a docstring. `summarize()` can be tested on one file.
- `if __name__ == "__main__":` means "only run this when the file is executed directly", so another script can import `summarize()` without the whole thing running.
- **Arguments** come from `sys.argv` (or the `argparse` module for anything more than one), with a sensible default. Nothing is hard-coded to your laptop.
- **An exit code:** `0` for success, non-zero for failure, so a scheduler knows whether it worked (Chapter 20).

Logging instead of printing

`print()` is fine while you're watching. A script that runs at 6 a.m. should write to a log:

```
import logging

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s",
datefmt="%H:%M:%S")
log = logging.getLogger("summary")
log.info("starting")
log.warning("3 rows could not be read")
print("(the log lines above go to stderr, so they don't mix with the report on stdout)")
```

(the log lines above go to stderr, so they don't mix with the report on stdout)

Logging gives you levels (`DEBUG`, `INFO`, `WARNING`, `ERROR`), timestamps, and the option to write to a file instead of the screen, all without changing the code that calls it.

Style, so other people can read it

Python's style guide (PEP 8) is worth ten minutes: four-space indents, `snake_case` names, spaces around operators, lines under about 100 characters, imports at the top. Don't memorize it: install a formatter (`ruff format` or `black`) and a linter (`ruff`), and let them do it. Most teams run them automatically, and consistent code is code you can read at speed.

17.15 Getting unstuck

Every programmer is stuck several times a day. What separates people is how quickly they get moving again.

1. **Read the error properly.** The last line names the problem; the lines above say where. Half of all "I'm stuck" moments end here.
2. **Check the obvious three:** am I in the right folder, the right environment, and running the file I think I am?
3. **Make the smallest failing example.** Cut everything unrelated. Often the answer appears while cutting.
4. **Search well.** Paste the error type and the meaningful part of the message, not your variable names: `python ValueError could not convert string to float csv` finds the answer; `my code doesn't work` doesn't. Prefer the official documentation and recent, high-voted answers; check the Python version an answer was written for.
5. **Read the documentation.** `help(str.strip)` in the REPL, or the module's page on docs.python.org. Library documentation usually has an examples section, which is where to start.
6. **Use an AI assistant like a knowledgeable colleague who can't see your data.** It's excellent at explaining an error, drafting a function, and suggesting an approach. It is confidently wrong often enough that you must run the code, check the numbers against something you trust, and never paste confidential data into it. If you can't explain what the code does, you can't defend the result, and you'll be the one in the meeting.
7. **Ask a human well:** what you're trying to do, what you did, what happened, what you expected, and the smallest example. Half the time you answer your own question while writing it.
8. **Take a break.** A ten-minute walk solves an astonishing share of bugs.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Installing packages into the system Python	Permission errors; one project breaks another	A virtual environment per project
Working in the wrong environment	<code>ModuleNotFoundError</code> for something you installed	Check the prompt shows <code>(.venv)</code> ; Python: Select Interpreter in VS Code

Mistake	Symptom	Fix
Expecting a script to print expressions	A script that "does nothing"	<code>print()</code> what you want to see
Mixing tabs and spaces	<code>IndentationError</code>	Let the editor insert four spaces
<code>=</code> where <code>==</code> belongs	<code>SyntaxError</code> , or a condition that's always true	<code>=</code> assigns, <code>==</code> compares
Forgetting that CSV values are text	<code>"430" + 5</code> fails, or totals concatenate	Convert with <code>float()</code> / <code>int()</code> , defensively
Assuming a column exists	<code>KeyError</code> in row 4,000	<code>.get()</code> with a default; check the header first
Opening files without an encoding	Gibberish for ₹ and Indian names on another machine	<code>encoding="utf-8"</code> every time
Building paths by gluing strings	Works on your laptop, fails on the server	<code>pathlib</code> , and <code>/</code> to join
Using <code>.sort()</code> and assigning the result	<code>None</code> where a list should be	<code>sorted(x)</code> returns; <code>x.sort()</code> mutates
Changing a list while looping over it	Items skipped for no visible reason	Build a new list, or loop over a copy
Bare <code>except:</code>	Typos and interrupts swallowed silently	Catch the specific errors you expect
Turning bad data into 0	A report that looks complete and is wrong	Count and report failures (Chapter 14)
Copying a formula into ten places	One of them is out of date within a month	A function, called ten times
One giant script with no functions	Impossible to test or reuse	Small functions with docstrings
Hard-coded paths and passwords	Runs only on your machine; secrets in Git	Arguments and environment variables
Notebook delivered as the product	Cells run out of order; nobody can schedule it	Move the working code into a script
Pasting AI-generated code without reading it	Plausible code, wrong numbers, no explanation	Run it, check against a number you trust, and understand it

In the real world: the fifteen-minute rescue

Riverstone's operations team sent Meera a folder on a Thursday afternoon: 36 CSV files, one per branch per month for the last quarter, exported from the old billing system before it was switched off. Finance wanted a single question answered by Monday: how much was invoiced in the quarter, by branch, and did it match the ERP?

Her first instinct was the usual one: open them in Excel, paste them together, pivot. She'd done it before with twelve files and it had taken an afternoon. Thirty-six files, and the columns weren't in the same order in every file.

She wrote fourteen lines of Python instead.

```
files = sorted(Path("billing_exports").glob("*.csv"))
rows = []
for path in files:
    with open(path, encoding="utf-8", newline="") as f:
        rows.extend(csv.DictReader(f))
```

The dictionary reader didn't care about column order, which solved the problem that would have cost her the afternoon. The script then converted amounts with a `to_float()` like the one in section 17.12, counted what it couldn't convert, grouped by branch with a `defaultdict`, and printed a table.

It took fifteen minutes to write and two seconds to run. It also found three things the spreadsheet approach would have hidden:

- **Eleven rows had an empty amount.** The script listed them with their file and line number; they turned out to be cancelled invoices the old system exported as blanks.
- **One file had 1,900 rows and the rest had about 400.** It was a duplicate export of September, saved twice under different names, and it would have silently added ₹1.4 crore to the quarter.
- **Two branches spelled their own name three ways.** The grouping showed "Kolkata", "kolkata", and "KOL" as separate branches, which is Chapter 14's mapping-table problem in miniature.

The quarter came to ₹4.7 crore less than Finance's first estimate, and the difference was entirely the duplicated file. Meera sent the script with the answer, so the next person could rerun it, and the team asked her to run it again for the two quarters before that. That took four seconds.

What made the difference:

- She **chose the tool for the shape of the job**: many files, uneven columns, a question that would be asked again.
- She **wrote defensively**: convert, count failures, report them, instead of letting bad rows vanish.
- She **kept the script**, so the work became reusable rather than a heroic afternoon.
- She'd have been slower on her first day of Python. That's the point of this chapter: the first script is slow, the tenth is faster than the spreadsheet, and the hundredth is a thing colleagues ask for by name.

Tools

- **Python 3.13 or 3.14** from python.org (3.10+ runs every example; the code in this chapter was run and checked on Python 3.12).
- **VS Code** with the Python and Jupyter extensions, or any editor you like. **JupyterLab** (`pip install jupyterlab`) for notebooks.
- **Standard library only**: this chapter installs nothing except Jupyter. `pandas` arrives in Chapter 18.
- **Companion files (`companion/ch17/`)**:
 - `build_ch17_files.py` builds everything below from the full dataset.

- `sales_exports/` : twelve monthly CSVs of the 24 key accounts' 2025 order lines (330 lines in total), plus `README.txt` so your folder code has to filter by extension.
 - `products.json` : the eight products as JSON.
 - `targets_2025.csv` : the key accounts' monthly targets for 2025 (₹4,240,000 for the year).
 - `broken_export.csv` : March's file with one unreadable row added, for section 17.12.
 - `summarize_exports.py` : the finished script from section 17.14 (also written by the chapter's own code).
 - **Optional:** `ruff` (formatter and linter), and the free official tutorial at docs.python.org for a second explanation of anything here.
-

The project: summarize a folder of exports

Goal: a script that reads a folder of CSV files and prints a summary of each, then writes a small report. This is the shape of a hundred real analyst tasks.

Option A: your own data. A folder of exports you receive regularly (anonymized).

Option B: Riverstone. `companion/ch17/sales_exports/`.

Steps

1. **List the files** with `pathlib`, ignoring anything that isn't a `.csv`.
2. **Read each file** with `csv.DictReader`.
3. **For each file, compute:** row count, the number of lines you could convert, the number you couldn't, net revenue excluding cancelled lines, and the number of distinct customers.
4. **Print a table** with aligned columns and a total row.
5. **Write a report** to `summary.md` : a Markdown table of the same numbers, with today's date and the folder name at the top.
6. **Handle the awkward cases:** an empty folder, a file with no rows, a file missing the `net_revenue` column, and a row whose value can't be converted. Report them; don't crash and don't silently skip them.
7. **Structure it properly:** small functions with docstrings, `if __name__ == "__main__":`, the folder as a command-line argument with a default, and an exit code.
8. **Check it:** the twelve Riverstone files hold **330 lines**, of which **326** are not cancelled, and their net revenue is **₹4,335,471.00** — the same figure as Chapters 10, 11, and 13, which is how you know the script is right.

Stretch goals

- Add a `--month 2025-10` option with `argparse` that summarizes one month.
- Add the targets from `targets_2025.csv` and show attainment per month (the year comes to 102.3% of ₹4,240,000).
- Write the summary as JSON as well, so another script could read it.

- Add a `tests.py` that checks `to_float("")`, `to_float("430.5")`, and `summarize()` on one known file (a first taste of Chapter 30's testing).
-

Timed challenge: the folder in forty minutes

Forty minutes, `companion/ch17/sales_exports/`, standard library only, no pandas. Answers at the end of the chapter.

- **Level 1:** How many CSV files are in the folder, and how many data rows in total?
 - **Level 2:** How many rows are not cancelled, and what is their total net revenue?
 - **Level 3:** Which month had the highest net revenue, and how much?
 - **Level 4:** What are the top three products by net revenue?
 - **Level 5:** How many distinct customers appear, and which two bought the most?
 - **Level 6:** Count the rows by status.
 - **Level 7:** What are the mean, median, and maximum line values (non-cancelled)?
 - **Bonus:** Total quantity sold, the largest single line quantity, and the number of cities.
-

You've got it when...

- ☐ You can install Python, create a virtual environment, and install a package into it without guessing.
 - ☐ You can explain the difference between the REPL, a notebook, and a script, and choose sensibly.
 - ☐ You use variables, f-strings, and the four basic types without looking them up.
 - ☐ You choose between a list, a dictionary, a tuple, and a set for a given job.
 - ☐ You write conditions and `for` loops, and can read a list comprehension aloud.
 - ☐ You write small functions with arguments, defaults, a return value, and a docstring.
 - ☐ You read and write CSV, JSON, and text files with `pathlib`, always specifying the encoding.
 - ☐ You read a traceback, name the ten common errors, and use `try / except` where it's the right answer.
 - ☐ You know what's in the standard library before reaching for a package.
 - ☐ You can turn a notebook that works into a script someone else can run, with arguments, logging, and an exit code.
 - ☐ When you're stuck, you have a routine, and it doesn't start with "ask someone".
-

Recap

- **Python earns its place** where a spreadsheet or SQL can't go: many files, APIs, schedules, statistics, and anything you'd otherwise repeat.
 - **Install it properly:** Python 3.13 or 3.14, on PATH; VS Code with the Python extension; a **virtual environment per project**, with `requirements.txt` recording what's in it.
 - **Types matter:** `str`, `int`, `float`, `bool`, `None`. CSV values arrive as text and must be converted, which is where cleaning bugs live. Floats are approximate.
 - **Collections:** list (ordered), tuple (fixed), dict (lookup), set (unique and fast membership). Counting and grouping with a dict, or `Counter` and `defaultdict`, is the manual version of a SQL `GROUP BY`.
 - **Conditions and loops** need a colon and consistent indentation; `for` over a collection is the normal loop; comprehensions are one-line loops that build a list.
 - **Functions** turn a formula into something you can name, reuse, test, and change in one place.
 - **Files:** `pathlib` for paths, `with open(...)` for reading and writing, `encoding="utf-8"` always, `csv.DictReader` and `json` for the two formats you'll meet most.
 - **Errors are information.** Read the last line first, know the common ten, catch what you expect, and count what you couldn't process instead of hiding it.
 - **The standard library** covers dates, counting, statistics, regular expressions, logging, and more before you install anything.
 - **A script** has a docstring, functions, arguments, logging, `if __name__ == "__main__":`, and an exit code. That's the difference between a notebook that worked once and a tool the team can run.
 - **Getting unstuck is a skill:** read the error, shrink the example, search precisely, use AI as a colleague you check, and ask humans well.
-

Practice exercises

Use `companion/ch17/` and the standard library only. Run everything; the point is the typing.

Warm-up

1. In the REPL, work out: `17 % 5`, `17 // 5`, `2 ** 8`, `int("42") + 8`, and `float("3.5") * 2`. What does `int("42.5")` do, and why?
2. Create variables for a customer name, an order count, and a revenue figure, then print one sentence with an f-string showing revenue to two decimals and orders as a whole number.
3. What's the difference between `"430" + "5"` and `430 + 5`, and how do you get 435 from the first pair?
4. Predict, then check: `bool("")`, `bool("0")`, `bool([])`, `bool([0])`, `None == False`, `1 == True`.
5. Given `cities = ["Mumbai", "Pune", "Bengaluru", "Delhi"]`, write expressions for: the first city, the last, the middle two, the list sorted, the list reversed, and whether "Kolkata" is in it.
6. Explain the difference between a list and a tuple, and give a Riverstone example of each.

Core

7. Write a function `net_line(quantity, unit_price, discount_pct=0)` that returns the line value, and a second function `is_large(value, threshold=25000)` that returns True or False. Test both on three lines you compute by hand.
8. Read `sales_exports/riverstone_2025_01.csv` with `csv.DictReader` and print the first three rows' customer name and net revenue. How many rows are there?
9. Total the non-cancelled net revenue in January. What is it, and how many lines did you count?
10. Loop over all twelve files and count the total number of rows in the folder. How many CSV files are there, and why must you filter by suffix?
11. Compute net revenue by month across the folder, print it as an aligned table, and name the highest and lowest months.
12. Build a dictionary of net revenue by product across the year, and print the top three.
13. Count the rows by status with `Counter`. How many are cancelled?
14. Using a set, list the distinct cities in the folder. How many are there?
15. Write `to_float(text, default=None)` and use it on `broken_export.csv`: how many rows convert, how many don't, and what are the bad values?
16. Read `products.json` and build a dictionary from product name to unit price. Which product is the most expensive, and what does it cost?
17. Read `targets_2025.csv` and calculate attainment per month (revenue ÷ target). In how many months did the key accounts beat their target, and what is the year's attainment?
18. Write a function `summarize(path)` that returns a dictionary with the file name, row count, non-cancelled line count, and net revenue. Run it on three files.
19. Write the folder summary to `summary.csv` with `csv.DictWriter`: one row per file, with a total row at the end.
20. Convert the `order_date` text to a `date` with `strptime`, and find the earliest and latest order dates in the folder.
21. Write a list comprehension that returns the net revenue of every cancelled line in the folder, then explain the same thing as a `for` loop.
22. Trigger each of these on purpose and paste the last line of the traceback: `KeyError`, `ValueError`, `FileNotFoundError`, `ZeroDivisionError`.

Stretch

23. Turn your folder summary into a script with `argparse`: `python summarize.py sales_exports --month 2025-10 --output summary.md`. Include a docstring, functions, and an exit code.
24. Add `logging` at INFO level to the script: one line when it starts, one per file processed, one warning per unreadable row, one line with the total at the end.
25. Write a function that takes the folder and returns a dictionary of customers whose net revenue exceeds a threshold, sorted from largest. Which customers exceed ₹400,000?
26. Write three small tests (plain `assert` statements in a `tests.py`) for `to_float` and `net_line`, and run them.

27. Read the folder once into a list of dictionaries, then answer exercises 11, 12, and 13 from that list instead of re-reading the files. Time both with `time.perf_counter()` and explain the difference.

Think about it

28. When is a notebook the right deliverable, and when is it the wrong one?
29. A colleague sends a script that works on their laptop and fails on yours with `FileNotFoundError` and then `ModuleNotFoundError`. What are the two likely causes, and what would you change in the script so it doesn't happen to the next person?
30. You ask an AI assistant for code to summarize a folder of CSVs. It gives you twenty lines that run and produce a number. What do you check before sending that number to Finance?

Key terms

program · Python · interpreter · REPL · script · notebook · Jupyter · VS Code · PATH · virtual environment (`venv`) · `pip` · PyPI · package · module · `import` · standard library · variable · assignment · `type ( str , int , float , bool , None )` · f-string · type conversion · floating-point approximation · list · index · slice · tuple · unpacking · dictionary · key and value · `get` with default · set · union, intersection, difference · condition · `if / elif / else` · comparison operator · truthiness · loop · `for` · `while` · `range` · `enumerate` · `zip` · `break` · `continue` · list comprehension · function · parameter · argument · default value · keyword argument · `return` · scope · docstring · `pathlib` · `glob` · context manager (`with`) · encoding · `csv.DictReader` · `csv.DictWriter` · JSON · serialization · exception · traceback · `try / except` · `Counter` · `defaultdict` · `datetime` · `strptime / strftime` · logging · `argparse` · `sys.argv` · exit code · `if __name__ == "__main__"` · PEP 8 · linter · formatter

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 18, Python for Analysts:** pandas replaces most of the loops in this chapter with whole-table operations, and adds Excel, SQL, APIs, and charts.
- **Chapter 19, Spreadsheet Automation:** the same programming ideas in VBA and Apps Script, for work that has to stay inside a spreadsheet.
- **Chapter 20, Automating Reports & Delivering Insights:** scheduling scripts, sending email, handling failure, and logging properly.
- **Chapter 21 and 22:** statistics, with simulations written in Python.
- **Chapter 26, Git:** version control for the scripts you're now writing.
- **Chapter 30, Python as Software, Not Scripts:** modules, packaging, testing, and type hints, once scripts grow up.
- **Interview preparation:** the Python & pandas Question Bank (Chapter 72) starts with exactly these fundamentals.

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. `17 % 5 = 2`; `17 // 5 = 3`; `2 ** 8 = 256`; `int("42") + 8 = 50`; `float("3.5") * 2 = 7.0`. `int("42.5")` raises `ValueError`: `int()` parses whole numbers only, so convert with `float()` first and then round or truncate deliberately.
2. For example: `print(f"{customer} placed {orders:,} orders worth ₹{revenue:,.2f}.")`
3. `"430" + "5"` concatenates to `"4305"`; `430 + 5` adds to 435. Convert first: `int("430") + int("5")`.
4. `bool("")` False; `bool("0")` **True** (a non-empty string); `bool([])` False; `bool([0])` True (a list with one item); `None == False` False; `1 == True` True (booleans are integers in Python).
5. `cities[0]`, `cities[-1]`, `cities[1:3]`, `sorted(cities)`, `cities[::-1]` (or `list(reversed(cities))`), `"Kolkata"` in `cities`.
6. A list can be changed (append, remove, sort in place); a tuple can't. List: the order lines in a file. Tuple: a (city, region) pair, or the (count, total, average) returned by `summarize()`.
7. For example, `net_line(45, 430, 5)` is 18,382.50; `net_line(10, 290)` is 2,900.00; `is_large(18382.5)` is False with the default threshold and True with 15,000.
8. January has **15** rows. The first three are Patel Kitchenware (₹2,900.00), Green Leaf Hotels (₹20,900.00), and Metro Mart (₹2,900.00).
9. **₹202,640.00** across 15 lines (January has no cancelled lines).
10. **12** CSV files, **330** data rows. `sales_exports` also contains `README.txt`; `glob("*.csv")` filters by suffix, and reading the README as CSV would produce nonsense rows rather than an error, which is worse.
11. Highest **October, ₹681,070.75**; lowest **June, ₹186,928.00**. (Full year, non-cancelled: Jan 202,640 · Feb 253,664 · Mar 278,008 · Apr 210,282 · May 329,359 · Jun 186,928 · Jul 232,692 · Aug 329,282 · Sep 558,315 · Oct 681,071 · Nov 633,408 · Dec 439,824.)
12. Storage Box 25L **₹908,212.50**, Storage Box 10L **₹860,946.00**, Industrial Crate **₹795,830.00**.
13. Delivered 322, Cancelled **4**, Shipped 3, Pending 1 (330 rows).
14. **16** cities.
15. 18 of the file's 19 rows convert; **one** row fails, the planted line with `quantity` 'twenty' and an empty `net_revenue`. Report both bad values with the line number, as section 17.12 does.
16. The most expensive product is the **Industrial Crate at ₹1,400.00**.
17. The key accounts beat their target in **6** of 12 months, and the year came to **102.3%** of ₹4,240,000 (₹4,335,471.00), the same figure as Chapter 10.
18. For example, January returns `{"file": "riverstone_2025_01.csv", "rows": 15, "lines_counted": 15, "net_revenue": 202640.0}`.

19. Use `csv.DictWriter` with `fieldnames=["file", "rows", "lines_counted", "net_revenue"]`, `writeheader()`, a row per file, and a final row with `file="TOTAL"` and the sum ₹4,335,471.00.
20. Earliest **2 January 2025**, latest **23 December 2025**, using `datetime.strptime(row["order_date"], "%Y-%m-%d").date()`.
21. `[float(r["net_revenue"]) for r in rows if r["status"] == "Cancelled"]` gives the four cancelled line values. The loop version creates an empty list, loops over `rows`, checks the status, and appends: same result, three more lines, and easier to extend when the condition grows.
22. `KeyError: 'city' · ValueError: could not convert string to float: '' · FileNotFoundError: [Errno 2] No such file or directory: 'nope.csv' · ZeroDivisionError: division by zero.` (The exact wording can differ slightly between Python versions.)
23. The script needs a module docstring, `argparse` with a positional `folder` and optional `--month` and `--output`, functions for reading and summarizing, `if __name__ == "__main__":`, and `raise SystemExit(main(args))` so the exit code reaches the shell.
24. `logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")`, then `log.info("reading %s", path)` per file and `log.warning("row %s unreadable: %r", line_no, value)` for bad rows. Use `%s` placeholders rather than f-strings in logging calls, so the formatting happens only if the message is actually emitted.
25. Two customers exceed ₹400,000 in 2025: **Sharma Hardware ₹502,775.00** and **Harbour Traders ₹412,680.50**.
26. For example: `assert to_float("430.5") == 430.5`, `assert to_float("") is None`, `assert net_line(10, 290) == 2900`. Run with `python tests.py`; silence means they passed, and an `AssertionError` names the failing line.
27. Reading once into memory is much faster, because the cost is opening and parsing the files, not the arithmetic. With 330 rows the difference is milliseconds; with 3 million rows, re-reading for every question is the difference between seconds and minutes. It's the same principle as Chapter 13's "let the database do the work once" and Chapter 16's Import mode.
28. A notebook is right for exploration, teaching, and showing your working, where the reader wants to see code and output together. It's wrong as a scheduled job, as a thing colleagues run, or as anything whose correctness depends on cells being run in order. Deliver a script (or a report the script produces) and keep the notebook as evidence.
29. `FileNotFoundError`: a hard-coded absolute path, or a relative path that assumes a particular working folder. `ModuleNotFoundError`: a package installed in their environment and not in yours, with no `requirements.txt`. Fix both: take paths as arguments with sensible defaults, resolve them with `pathlib`, and ship a `requirements.txt` (and a one-line "how to run" in the docstring).
30. Read it line by line and make sure you can explain each one. Check what it excludes (cancelled rows? blank amounts?), what it does with rows it can't parse, and whether it counts files you didn't intend (the README). Then reconcile the output against something you trust: one file totalled by hand, or the same period from the ERP. The number goes to Finance under your name, not the assistant's.

Timed challenge answers. Level 1: **12** files, **330** rows. Level 2: **326** non-cancelled rows, **₹4,335,471.00**. Level 3: **October**, **₹681,070.75**. Level 4: Storage Box 25L ₹908,212.50 · Storage Box 10L ₹860,946.00 · Industrial Crate ₹795,830.00. Level 5: **23** customers with non-cancelled revenue; **Sharma Hardware ₹502,775.00** and **Harbour Traders ₹412,680.50**. Level 6: Delivered 322 · Cancelled 4 · Shipped 3 · Pending 1. Level 7: mean **₹13,298.99**, median **₹10,212.50**, maximum **₹56,700.00**. Bonus: **9,475** units, largest line **85** units, **16** cities.